Kauno technologijos universitetas
Informatikos fakultetas

Aplinkos kompiuterizacija ir protingos sistemos (T120B139)

Laboratorinių darbų ataskaita

Gintarė Jasiukevičiūtė IFB-3

Studentė

lekt. PLAUSKA Ignas

Dėstytojai

Kaunas 2025

TURINYS

1. Praktinis darbas Nr. 1 (EasyEDA įvadas PR0) 4

1.1. Darbo tikslas 4

1.2. Rezultatų aprašymas 4

1.3. Išvados 5

2. Laboratoriniai darbai PR1-PR6 6

2.1. Praktinis darbas Nr. 2 (PR1. Prisijungimas prie virtualios mašinos ir jos paruošimas) 6

2.1.1 Darbo tikslas 6

2.1.2 Rezultatų aprašymas 6

2.1.3 Išvados 7

2.2. Praktinis darbas Nr. 3 (PR2. Paprasto įrenginio sukūrimas) 8

2.2.1 Darbo tikslas 8

2.2.2 Rezultatų aprašymas 8

2.2.3 Išvados 9

2.3. Praktinis darbas Nr. 4 (PR3. Apšvietimo valdymas) 10

2.3.1 Darbo tikslas 10

2.3.2 Rezultatų aprašymas 10

2.3.3 Išvados 12

2.4. Praktinis darbas Nr. 5 (PR4. Kambario šildymo projektas) 13

2.4.1 Darbo tikslas 13

2.4.2 Rezultatų aprašymas 13

2.4.3 Išvados 16

2.5. Praktinis darbas Nr. 6 (PR5. Įrenginių virtualizavimas) 17

2.5.1 Darbo tikslas 17

2.5.2 Rezultatų aprašymas 17

2.5.3 Išvados 19

2.6. Praktinis darbas Nr. 7 (PR6. Namo šildymo projektas) 20

2.6.1 Darbo tikslas 20

2.6.2 Rezultatų aprašymas.....21
2.6.3 Išvados.....26
3. Bendros išvados.....27

1. Praktinis darbas Nr. 1 (EasyEDA įvadas PR0)

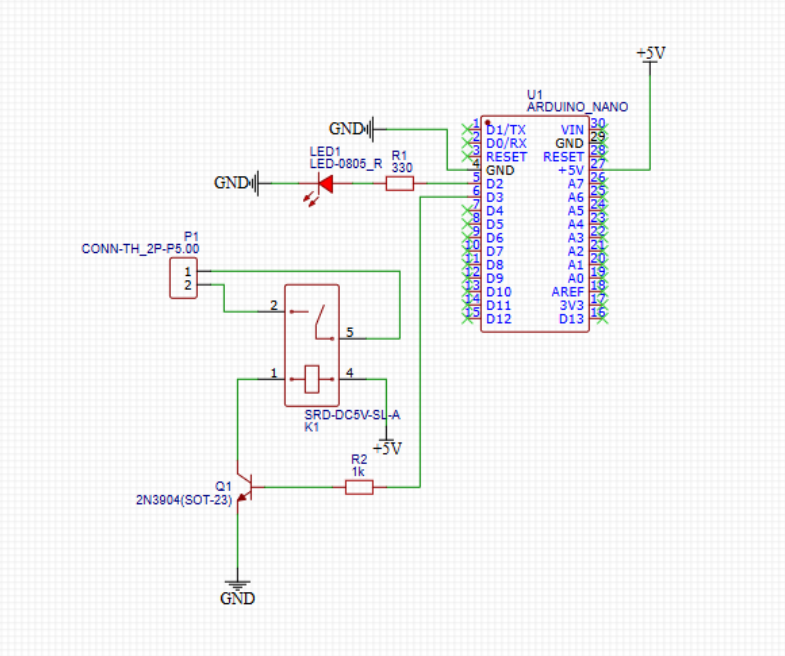
1.1. Darbo tikslas

Šio darbo tikslas – susipažinti su **EasyEDA** sistema – internetine elektronikos projektavimo aplinka, leidžiančia kurti elektrines schemas, projektuoti spausdintinių plokščių (PCB) išdėstymą. Darbo metu bus apžvelgtos pagrindinės EasyEDA funkcijos, vartotojo sąsaja, komponentų biblioteka ir bandymas sukurti paprastą elektroninę schemą su Arduino Nano, rele, LED'u ir tranzistoriumi.

1.2. Rezultatų aprašymas

Turėjau sukurti schemą su tokiais komponentais:

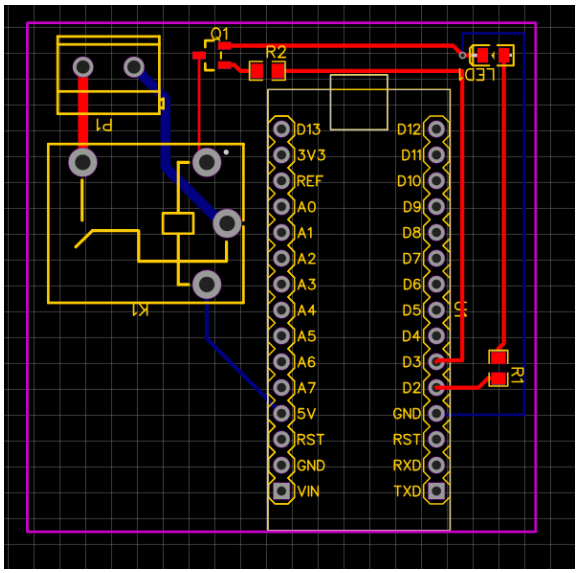
	Nr	Elemento kodas	Footprint
Relė	K1	SRD-DC5V-SL-A	RELAY-TH_SRD-DCXXV-SL-A
LED'as	LED1	LED-0805_R	LED0805_RED
Kontaktai	P1	CONN-TH_2P-P5.00	CONN-TH_2P-P5.00
Tranzistorius	Q1	2N3904(SOT-23)	SOT-23-3_L2.9-W1.3-P1.90-LS2.4-TR
Varža	R1	330	R0805
Varža	R2	1k	R0805
Mikrovaldiklis	U1	ARDUINO_NANO	COMM-TH_ARDUINO_NANO



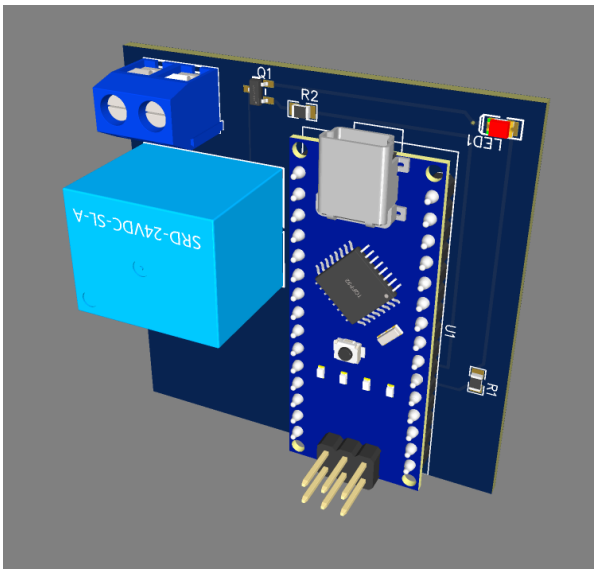
Pav 1 Projektuojama schema

Schema pasidariau pagal duotą pavyzdį, ir tuomet ja konvertavus į PCB plokštės schemą reikėjo kuo optimaliau viska sutalpinti – viskas išdėsčius turi tilpti į plokštę ir takai negali kirstis. Be to svarbus funkcionalumas, pavyzdžiui negali būti užblokuoti laidams ar kitiems prietaisams prijungti skirtos vietos.

Gavosi toks variantas:



Pav 2 PCB plokštės schema



Pav 3 PCB plokštė

1.3. Išvados

EasyEDA yra gana patogi aplinka, viskas per naršyklę, nereikia nieko įdiegti. Užduotį atlikti pavyko gana greitai ir lengvai. Sunkiausia buvo sugalvoti ir suprojektuoti kaip viskas estetiškai ir optimaliai tilps į plokštę.

2. Laboratoriniai darbai PR1-PR6

2.1. Praktinis darbas Nr. 2 (PR1. Prisijungimas prie virtualios mašinos ir jos paruošimas)

2.1.1 Darbo tikslas

Šiame darbe turėjome prisijungti prie mums sukurtos virtualios mašinos, bei joje pasiruošti aplinką tolesniam darbui.

Laukiamas rezultatas:

- Prisijungta prie virtualios mašinos (VM) per SSH terminalą (**PuTTY** ar kitą).
- VM paleisti **HomeAssistant, ESPHome, mosquitto** Docker konteineriai.
- Galima prisijungti prie **HomeAssistant** ir **ESPHome Device Builder** tinklapių ir jais naudotis.
- Tinklapiams nustatyti slaptažodžiai.
- **HomeAssistant** pridėta mqtt integracija su vietiniu mqtt servisu, jo veikimas išbandytas.

2.1.2 Rezultatų aprašymas

Prisijungus prie savo gautos virtualios mašinos (man suteiktas IP buvo 158.129.30.142) įsirašiau HomeAssistant ir ESPHome dockerius į /root/AKIPS2025 aplanką bei įkėliau konteinerių konfigūracijos failą docker-compose.yaml:

```
services:
  homeassistant:
    container_name: homeassistant
    image: ghcr.io/home-assistant/home-assistant:stable
    volumes:
      - /root/AKIPS2025/appdata/homeassistant:/config
    environment:
      - TZ=Europe/Vilnius
    restart: unless-stopped
    network_mode: host

  esphome:
    container_name: esphome
    image: ghcr.io/esphome/esphome
    volumes:
      - /root/AKIPS2025/appdata/esphome/config:/config
      - /etc/localtime:/etc/localtime:ro
    environment:
      - USERNAME=gintare
      - PASSWORD= [mano slaptažodis]
    restart: unless-stopped
    privileged: true
    network_mode: host
```

Tuomet naršyklėje jau galėjau pasiekti savo ESPHome ir HomeAssistant puslapius suvedus savo virtualios mašinos IP su šiais portais:

- **HomeAssistant**: 8123
- **ESP Home Device Builder:** 6052

Tada reikėjo paleisti Mosquitto MQTT servisą:

Kataloge /root/AKIPS2025/appdata sukuriau naują katalogą *mosquitto* ir tada jame - *config* katalogą kuriame sukūriau failą mosquitto.conf:

```
port 1883

allow_anonymous true

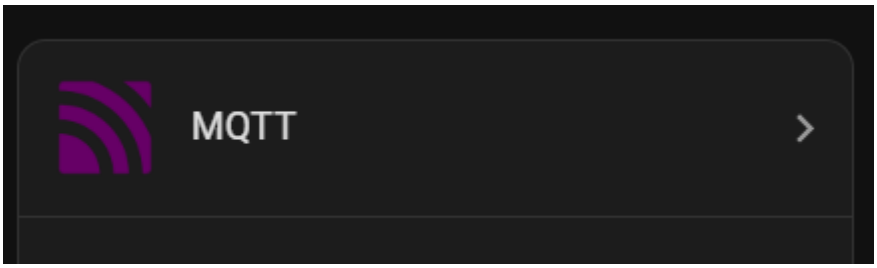
persistence true
persistence_file mosquitto.db
persistence_location /mosquitto/data

log_dest file /mosquitto/log/mosquitto.log
log_type all
```

Papildžiau docker-compose.yaml:

```
mosquitto:
        image: eclipse-mosquitto:2
        container_name: mosquitto
        restart: unless-stopped
        network_mode: host
        volumes:
        - /root/AKIPS2025/appdata/mosquitto/config:/mosquitto/config
        - /root/AKIPS2025/appdata/mosquitto/data:/mosquitto/data
        - /root/AKIPS2025/appdata/mosquitto/log:/mosquitto/log
```

Integravus MQTT į HomeAssistant jau galėjau prie „devices“ matyti jo skiltį, kur vėliau rasiu savo įrenginius:



Pav 4 MQTT sąsaja HA lange

2.1.3 Išvados

Sekdama visus Moodle sistemoje pateiktus žingsnius, sėkmingai pasileidau virtualią mašiną, bei dockerius ir MQTT integraciją, kuri paruošia mane tolimesniam darbui su įrenginiais.

2.2. Praktinis darbas Nr. 3 (PR2. Paprasto įrenginio sukūrimas)

2.2.1 Darbo tikslas

Šiame darbe, naudodami **ESPHome** aplinką turite sukurti paprastą įrenginį, ir jį integruoti į **HomeAssistant**.

Laukiamas rezultatas:

- **ESPHome** aplinkoje sukurto įrenginio kodas užprogramuotas į M5 stack valdiklį.
- Įrenginys integruotas į **HomeAssistant**, panaudojant MQTT.
- Spaudant mygtukus (A, B, C) M5 stack valdiklyje, jų būsena (ON ar OFF) matoma **HomeAssistant** pagrindiniame puslapyje (Dashboard).

2.2.2 Rezultatų aprašymas

Pirmiausiai reikėjo sukurti savo naujo demo įrenginio aprašą per ESPHome ir aprašyti tris M5Stack valdiklio mygtukus A, B ir C:

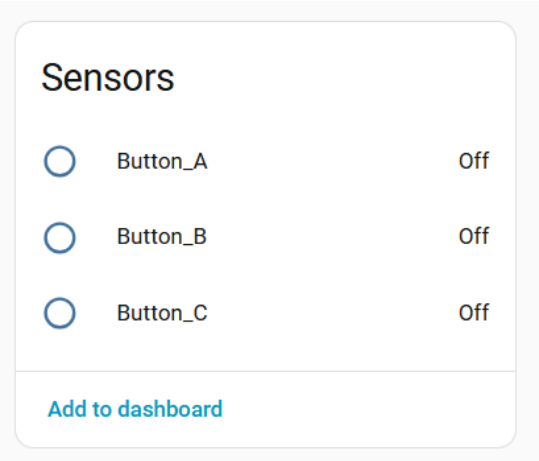
```
mqtt:
  broker: 158.129.30.142
  port: 1883
  discovery_unique_id_generator: mac
  discovery_object_id_generator: device_name
  custom_mac: 30:AE:A4:F5:A4:5C

text_sensor:
  - platform: wifi_info
    ip_address:
      name: "IP Address"
    mac_address:
      name: "Real MAC"

binary_sensor:
  - platform: gpio
    name: "Button_A"
    pin:
      number: GPIO39
      inverted: true
      mode:
        input: true
  - platform: gpio
    name: "Button_B"
    pin:
      number: GPIO38
      inverted: true
      mode:
        input: true
  - platform: gpio
    name: "Button_C"
    pin:
```

```
number: GPIO37
inverted: true
mode:
  input: true
```

Validavus ir instaliavus kodą su šiuo papildymų, galėjau matyti savo įrenginį per HomeAssistant ir jo mygtukų besikeičiančias būsenas po paspaudimo.



Pav 5 Mygtukų vaizdas per HomeAssistant

2.2.3 Išvados

Laukiamą rezultatą gauti pavyko, integravau M5Stack valdiklį į HomeAssistant ir galėjau matyti kaip keičiasi mygtukų būsenos po paspaudimo. Likusius du mygtukus integruoti savarankiškai buvo lengva.

2.3. Praktinis darbas Nr. 4 (PR3. Apšvietimo valdymas)

2.3.1 Darbo tikslas

Šiame darbe papildysiu PR2 sukurtą įrenginį dviem naujais komponentais:

- **switch** - paprasčiausias jungiklis, kuris valdys šviestuvą (išbandysime su LED).
- **light** - keičiamo intensyvumo šviestuvas (išbandysime su LED).

Taip pat, sukursiu jiems valdymo mygtukais taisykles.

Laukiamas rezultatas:

- PR2 darbe sukurtas įrenginys papildytas switch ir light komponentais, kurie valdo prie **M5stack** valdiklio GPIO prievadų prijungtus LED'us.
- Šie komponentai matomi įrenginio apraše **HomeAssistant** aplinkoje.
- Taip pat **HomeAssistant overview** skiltis papildyta naujus komponentus atitinkančiais grafiniais komponentais.
- Sukurtos taisyklės (**automation**):
    - Spaudant A mygtuką, keičiama paprasto **LED** šviestuvo būsena (įjungiama/išjungiama).
    - B ir C mygtukais keičiamas **LEDC** šviestuvo galingumas.
    - Abu šviestuvai užgesinami praėjus 10 sekundžių po jų įjungimo.

2.3.2 Rezultatų aprašymas

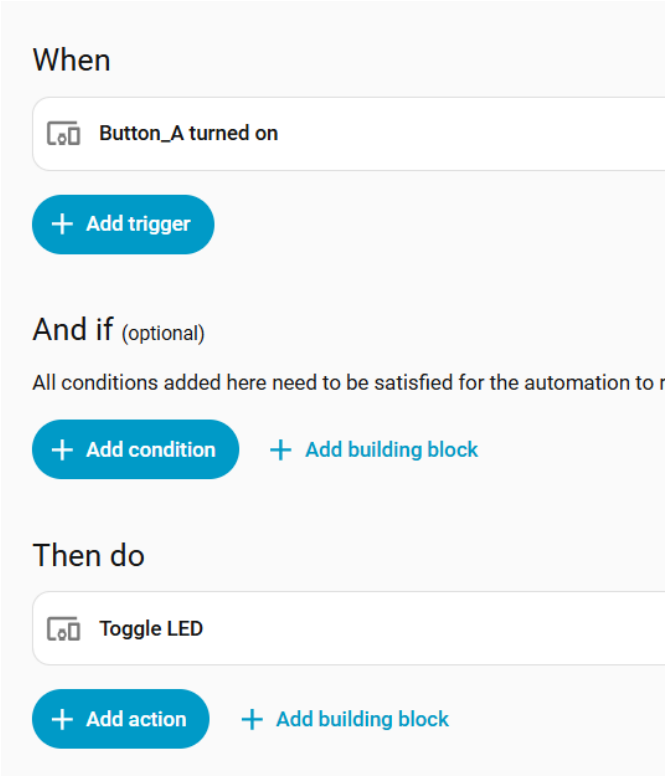
Naudodamasi dokumentacija, praeito praktinio darbo metu sukurtą yaml failą papildžiau tokiu kodu, kad sukurti jungiklį ir LED šviestuvo komponentus:

```
switch:
  - platform: gpio
    pin:
      number: GPIO23
      inverted: True
    name: "LED"

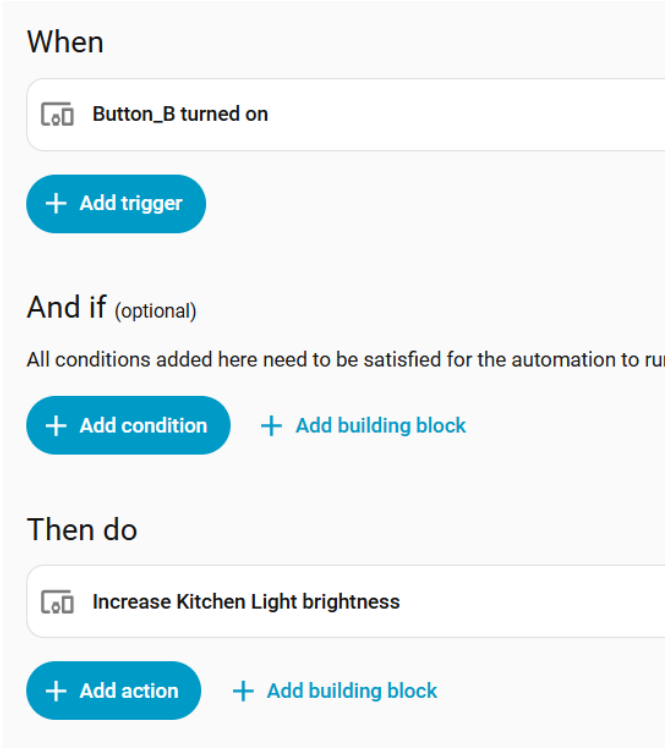
output:
  - platform: ledc
    pin: GPIO17
    id: gpio_17

light:
  - platform: monochromatic
    output: gpio_17
    name: "Kitchen Light"
```

Tuomet per HomeAssistant reikėjo sukurti jiems Automation taisykles:

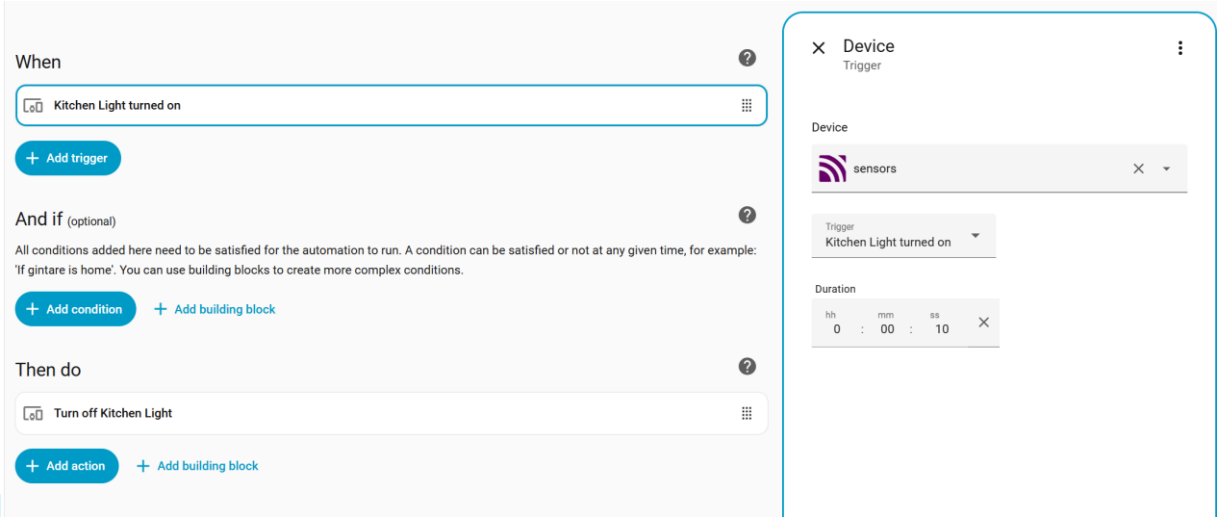


Pav 6 įjungimo ir iškungimo taisyklė



Pav 7 Šviesos ryškinimo taisyklė

Analogiškai su C mygtuku buvo sukurta ir šviesos galingumo mažinimas.



Pav 8 Automatinio išsijungimo taisyklė

2.3.3 Išvados

Šio praktinio darbo užduotį pavyko lengvai įgyvendinti, kadangi šitoms taisyklėms kurti buvo naudojamas tiesiog HomeAssistant taisyklių šablonas, o patys komponentai buvo nukopijuoti iš dokumentacijos.

2.4. Praktinis darbas Nr. 5 (PR4. Kambario šildymo projektas)

2.4.1 Darbo tikslas

Laukiamas rezultatas:

- Sukurtas naujas įrenginys (naudojant M5Stack) - termostatas, valdantis kambario šildymą:
  - Termostatas naudoja temperatūros daviklį **SHT30**.
  - Mygtukų paspaudimais galima:
    - Įjungti/išjungti šildymo veikimą
    - Didinti mažinti norimą temperatūrą.
- Taip pat įrenginys gali valdyti kambario apšvietimą.
  - Apšvietimas valdomas tik iš **HomeAssistant** puslapio.
- HomeAssistant sukurta nauja kortelė "Šildymas", kurioje matoma ir galima valdyti termostato bei apšvietimo būsenas.

2.4.2 Rezultatų aprašymas

Naudojantis dokumentacija buvo sukurtas Thermostat Climate Controller komponentas ir aprašytas yaml faile, be to pridėtas temperatūros sensorius. Pildžiau prieš tai sukurtą failą, tik šiuo atveju Switch atliko šildytuvo „heater“ funckiją:

```
switch:
  - platform: gpio
    id: heater
    pin:
      number: GPIO23
      inverted: True
    name: "heater"
output:
  - platform: ledc
    pin: GPIO17
    id: gpio_17

light:
  - platform: monochromatic
    output: gpio_17
    name: "Kitchen Light"


climate:
  - platform: thermostat
    name: "Thermostat Climate Controller"
    sensor: my_temperature_sensor
    min_heating_off_time: 30s
    min_heating_run_time: 30s
    min_idle_time: 30s
    heat_action:
      - switch.turn_on: heater
```

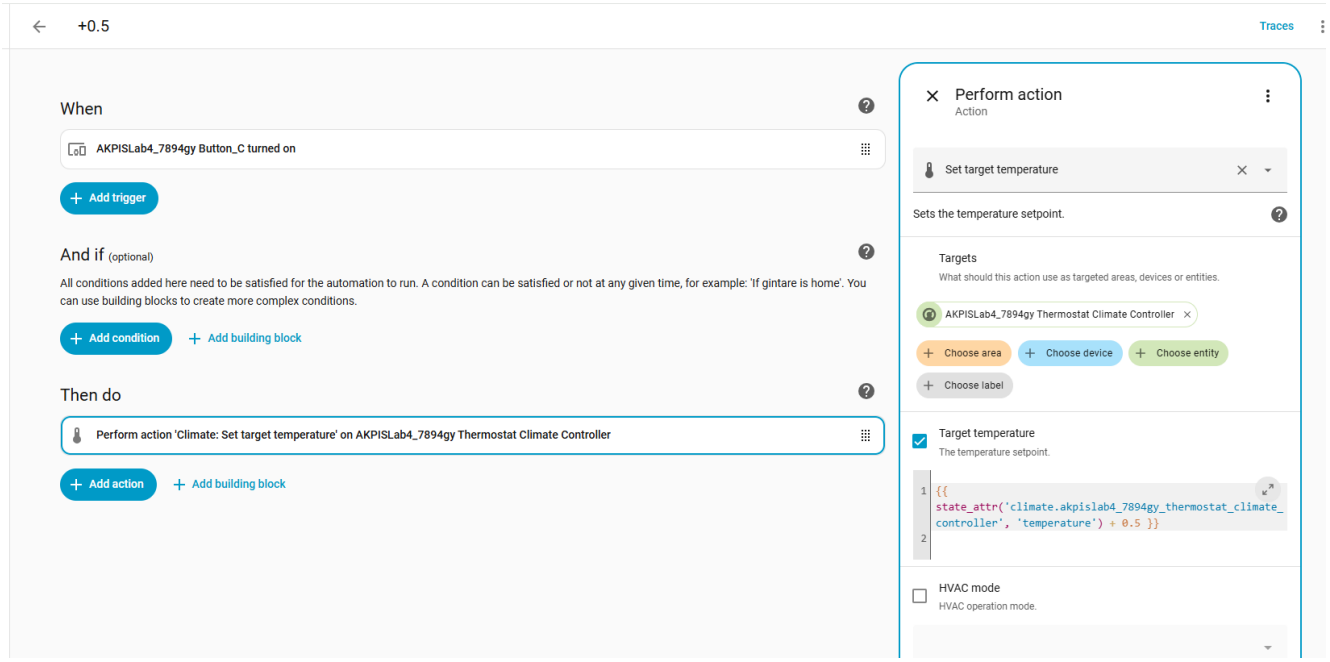
```
    idle_action:
      - switch.turn_off: heater

    default_preset: Home
    preset:
      - name: Home
        default_target_temperature_low: 20 °C
i2c:
  - id: bus_a
    sda: GPIO21
    scl: GPIO22

sensor:
  - platform: sht3xd
    i2c_id: bus_a

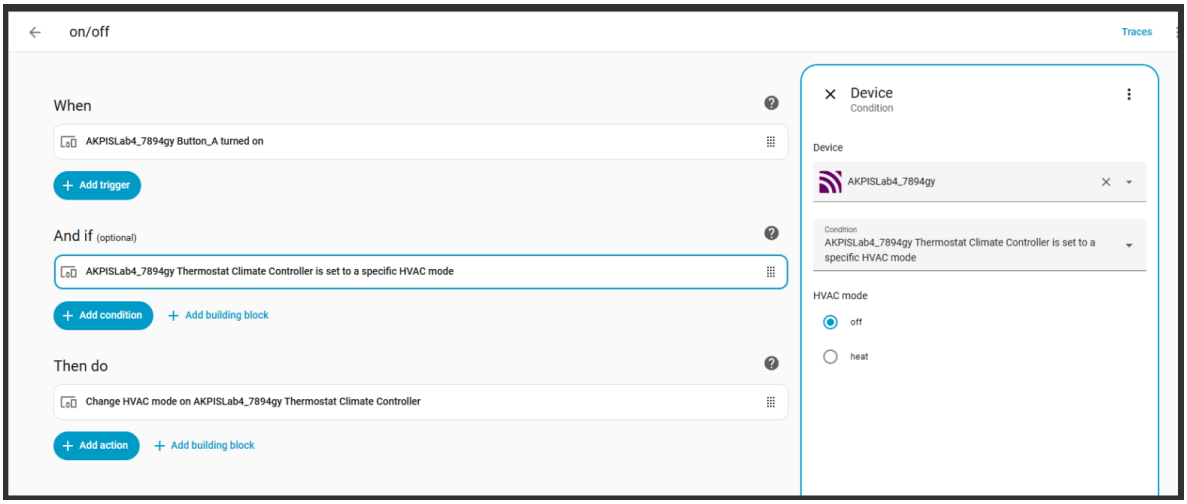
    temperature:
      name: "Living Room Temperature"
      id: my_temperature_sensor
    humidity:
      name: "Living Room Humidity"
    address: 0x44
    update_interval: 5s
```

O tada naudojantis jinja išraiškomis aprašiau automatizavimo taisykles per HomeAssistant:

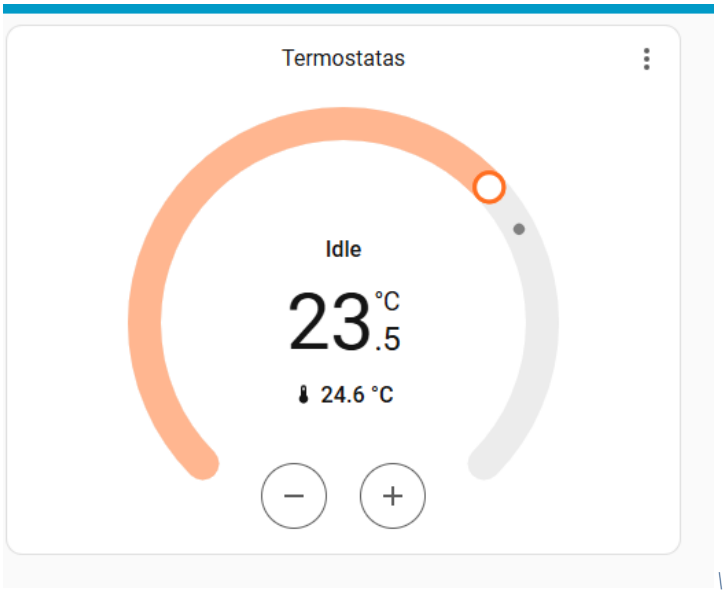


Pav 9 Temperatūros didinimo taisyklė

Analogiškai buvo sukurta ir temperatūros mažinimo taisyklė tik naudojau mygtuką B. Šildymo veikimui įjungimo/išjungimo funkcijai buvo panaudotas A mygtukas su paprastu HomeAssistant taisyklių šablonu:



Pav 10 Šildymo įjungimas



Pav 11 Termostato HA kortelė



Pav 12 Termostatas šyla kai temperatūra per maža

Overview lange buvo sukurta nauja kortelė, kurioje buvo galima stebėti termostato veikimą ir jo būsenų pasikeitimą priklausomai nuo mygtukų paspaudimo bei esamos auditorijos temperatūros.

2.4.3 Išvados

Šį skyrių atlikti buvo šiek tiek sunkiau, kadangi teko naudotis beveik vien tik dokumentacija ir pačiai išsiaiškinti kaip sukurti ir aprašyti atitinkamus komponentus, tačiau galiausiai viską padaryti pavyko ir buvo patobulinti naudojimosi dokumentacija įgūdžiai. Kilo sunkumų su jinja išraiškomis, nes taisyklėms aprašyti šablonų nebeužteko.

2.5. Praktinis darbas Nr. 6 (PR5. Įrenginių virtualizavimas)

2.5.1 Darbo tikslas

Laukiamas rezultatas:

- Sukurtas virtualus temperatūros sensorius **M5 Stack** valdiklyje:
  - Naudoja SHT30 temperatūrą, kas 10s ją padidinamas 0.5 laipsnio.
- Sukurtas virtualus temperatūros sensorius **HomeAssistant**, panaudojant **HomeAssistant Community Store** ir **Virtual Components**.
  - Taip pat naudoja SHT30 temperatūrą, kas 10s ją padidinamas 1 laipsniu.

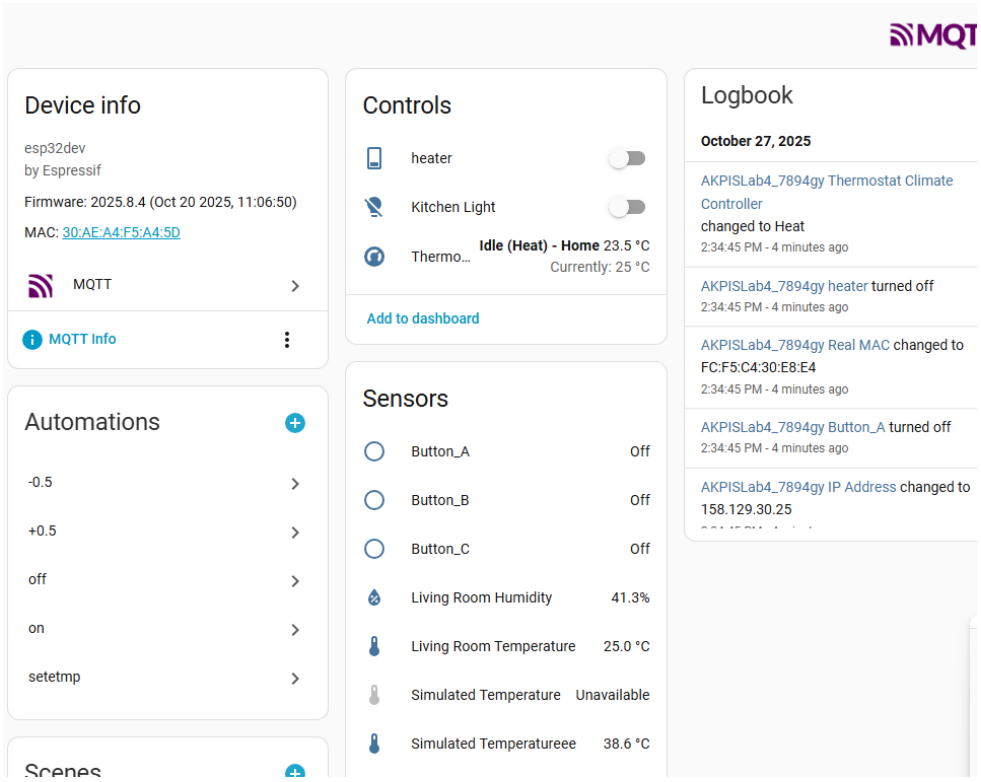
2.5.2 Rezultatų aprašymas

Turimą yaml kodą papildžiau su tokiu virtualaus sensoriaus aprašymu:

```
- platform: template
  id: simulated_temperature
  name: "Simulated Temperatureee"
  device_class: temperature
  unit_of_measurement: "°C"
  accuracy_decimals: 1
  update_interval: 10s
  lambda: |-
   float virt_temp = id(my_temperature_sensor).state + 0.5;

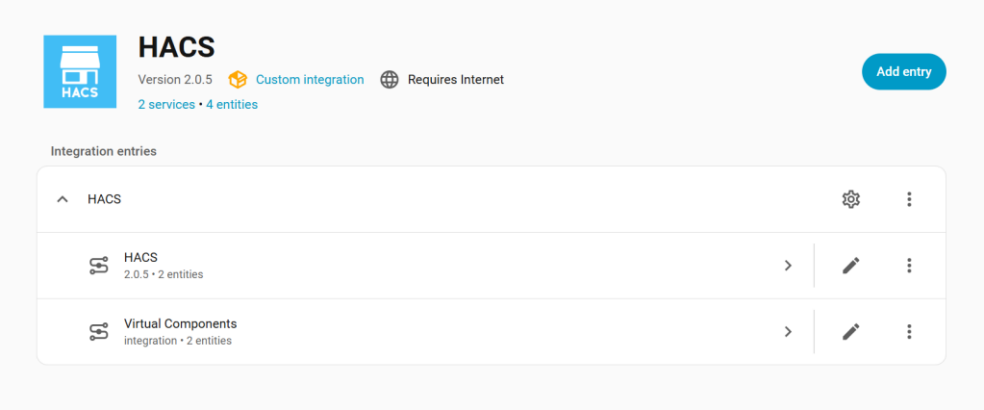
   if (id(simulated_temperature).state > id(my_temperature_sensor).state ) {
     float virt_temp2 = id(simulated_temperature).state + 0.5;
     return virt_temp2;
   } else {
     return virt_temp;
     }
```

Pirmą kartą kai įjungiamas termostatas yra paimama temperatūra iš tikro sensoriaus, o vėliau vis tikrinama ar ji jau buvo kelta ir jei buvo tai toliau ją keliam po pusę laipsnio kas 10 sekundžių.



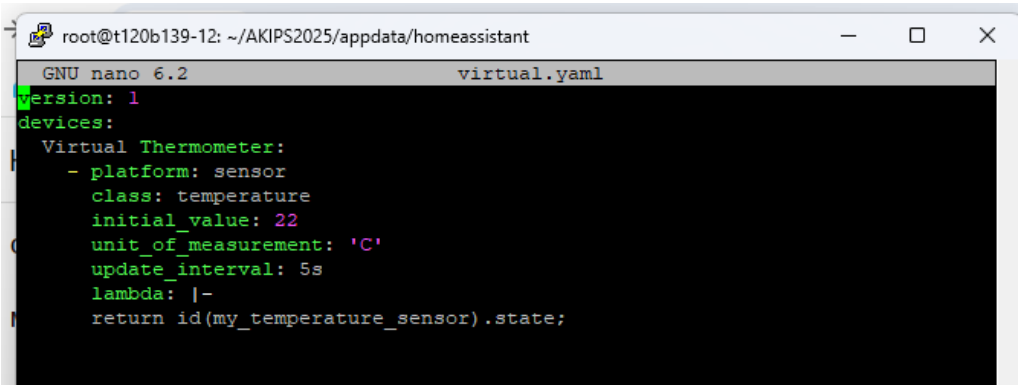
Pav 13 Įrenginio nustatymų langas

Tuomet įdiegiau HACS plėtinį, kad galėčiau sukurti virtualų komponentą, šiam tikslui pasiekti naudojausi oficialiu HACS įdiegimo aprašu.



Pav 14 HACS plėtinys per HA

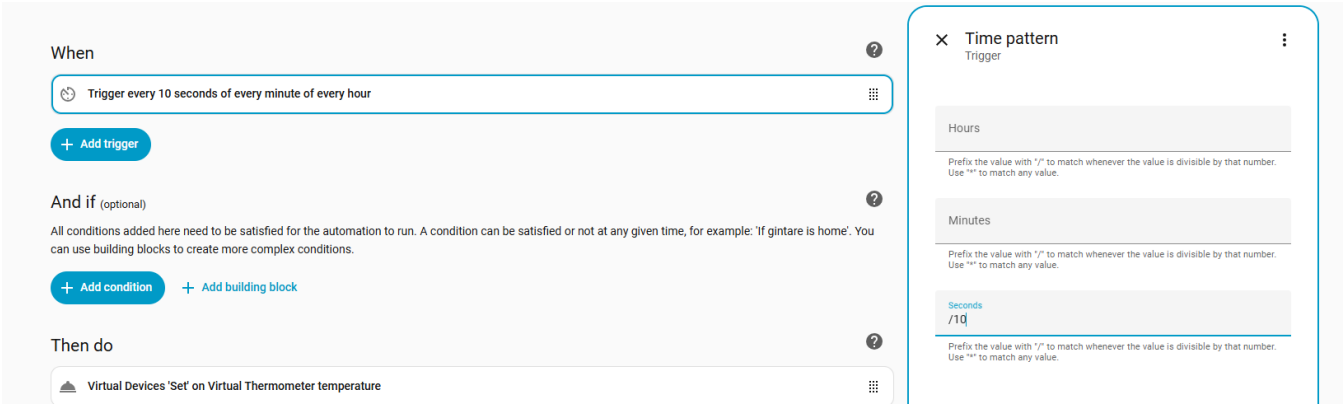
Per virtualią mašiną susikūriau virtual.yaml failą:



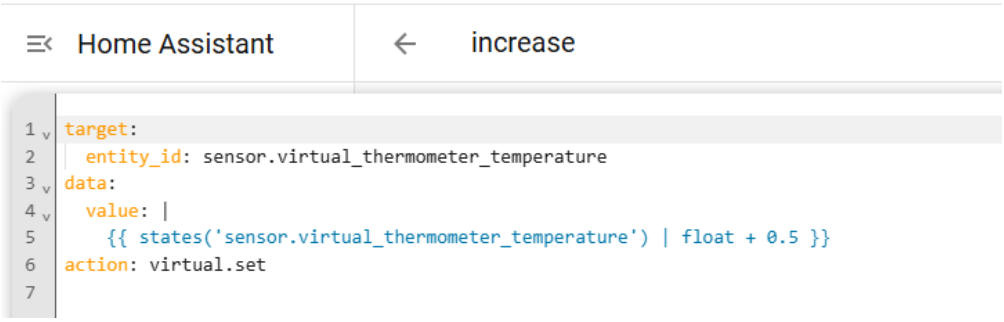
Pav 15 virtual.yaml

Ir tada per HomeAssistant jau galėjau matyti šį įrenginį, kuris paima temperatūrą iš fizinio įrenginio, naujoje „Virtual Components“ skiltyje.

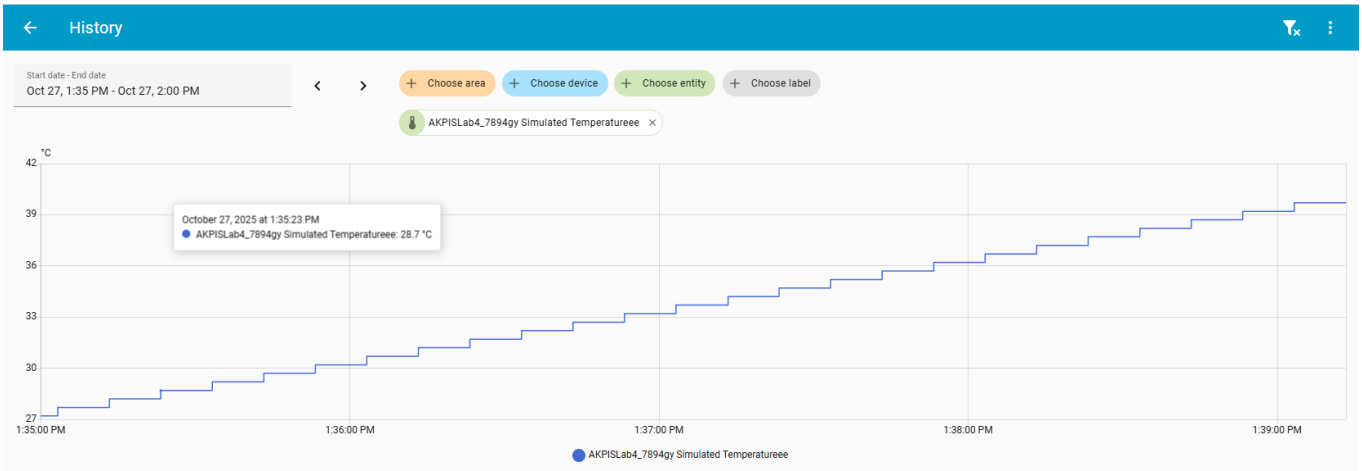
Beliko tik aprašyti taisykles kurios leis simuliuos automatinį temperatūros didinimą. Taip pat buvo naudojamos jinja išraiškos.



Pav 16 Taisyklės šablonas



Pav 17 Jinja išraiška temperatūros didinimui



Pav 18 Temperatūros kitimo grafikas

Pateiktame grafike matosi, kad Simuliuota temperatūra kyla pagal nurodytą laiko tarpą ir kiekį.

2.5.3 Išvados

Užduotį atlikti pavyko, buvo sukurtas simuliuotos temperatūros komponentas, kurio reikšmę buvo galima keisti, ne taip kaip fizinio komponento. Taip pat pavyko sukurti ir simuliuoti temperatūrą per Virtual Components plėtinį. Man šis būdas buvo daug patogesnis, yra daug šablonų.

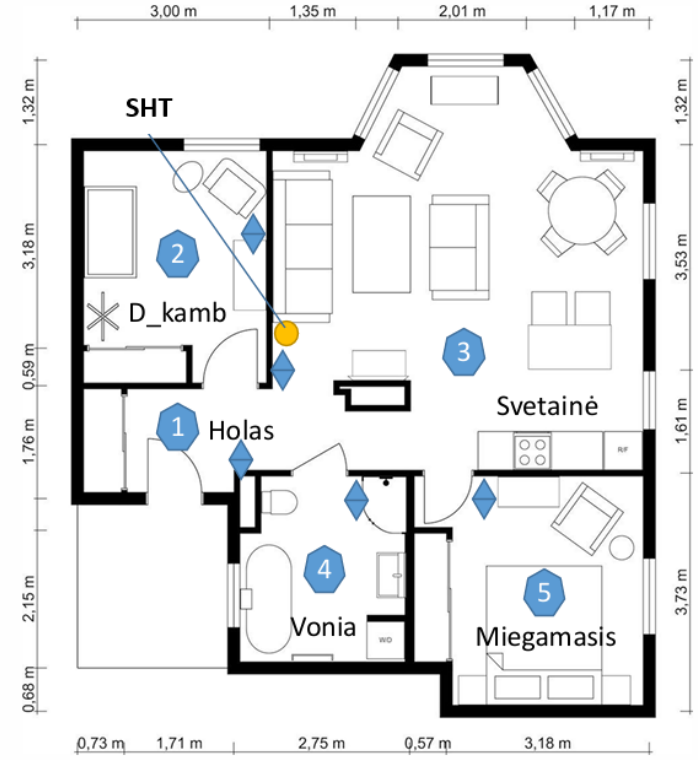
2.6. Praktinis darbas Nr. 7 (PR6. Namo šildymo projektas)

2.6.1 Darbo tikslas

Užduotis:

Pagal pateiktą planą sudaryti namo šildymo projektą. Čia:

- 🟡 - Tikras SHT30 sensorius prijungtas prie **M5 Stack** valdiklio.
- 🔹 - Simuliuojamas kambario termostatas.
- 1️⃣ - 5️⃣ - Simuliuojamas kambario šildytuvas.



- **HomeAssistant** sukurkite po patalpą kiekvienam kambariui (**Settings -> Areas, labels and zones**)
- Sukurkite 5 termostatus, iš kurių:
  - Svetainės termostatas įdiegtas M5 Stack modulyje su prijungtu SHT30 temperatūros davikliu. Šio termostato būseną ir norimą temperatūrą galima valdyti mygtukais M5 Stack modulyje.
  - Bent 1 termostatas simuliuojamas panaudojant virtualius **Virtual Devices** įrenginius (Sukurkite temperatūros sensoriaus ir jungiklio komponentus, o tada sukurkite termostatą per **Settings > Devices & services > Helpers**).
  - Bent 1 termostatas simuliuojamas su **Wokwi**
    - Wokwi temperatūros davikliui panaudokite DHT22
    - Aprašant ESPHome - model: DHT22_TYPE2
  - Kiti - laisvu pasirinkimu.
- Visi termostatai:
  - Pradinę temperatūrą nuskaito iš SHT30 daviklio.
  - Vėliau simuliuoja šildymą - jei kambario šildytuvas įjungtas - temperatūra kyla, jei išjungtas - krenta.
- Visus įrenginius ir aprašytas taisykles priskirkite konkretiems kambariams.

- Sudarykite skydelį **Overview** lange, kuriame matytųsi:
    - Visų kambarių termostatai, temperatūros ir šildytuvo būsenos istorijos.
    - Tikra temperatūra ir jos istorijos grafikas iš DHT30 daviklio.

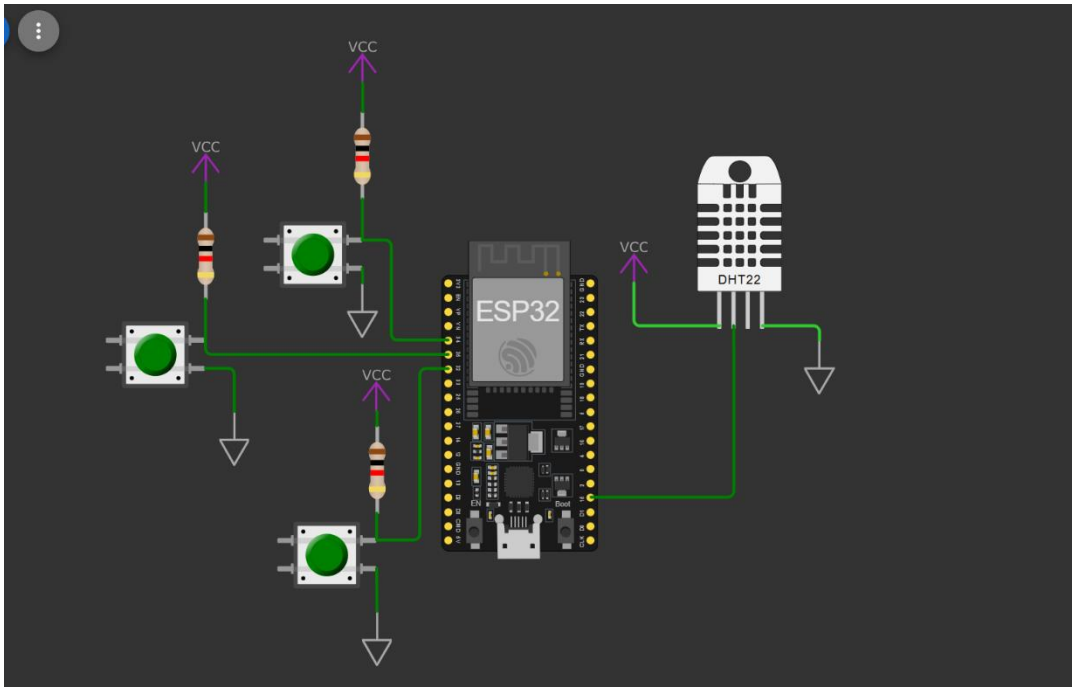
2.6.2 Rezultatų aprašymas

Svetainės termostatui buvo naudojamas jau prieš tai sukurtas ir aprašytas M5Stack įrenginys, miegamajam simuliavau wokwi įrenginį, o visi likę buvo virtualūs, nes juos lengviausia simuliuoti ir keisti jų temperatūrą.

Miegamojo termostato yaml kodas iš esmės buvo toks pats, tik naudojau kitus GPIO pins ir kitoks daviklio aprašas:

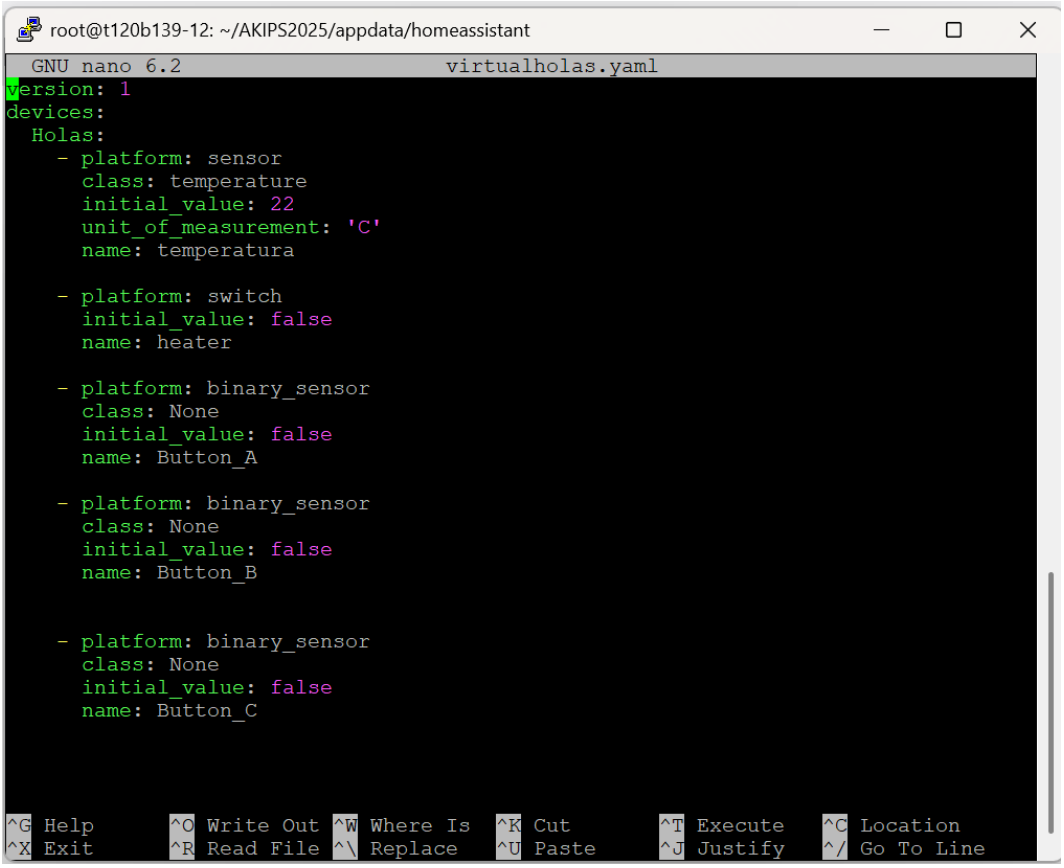
```
- platform: dht
  model: DHT22_TYPE2
  pin: GPIO15
  temperature:
    name: "Miegamojo temperatura"
    id: bedroom_temperature_sensor
  update_interval: 5s
```

Wokwi simuliuojamo įrenginio schema atrodo taip:



Pav 19 Wokwi schema

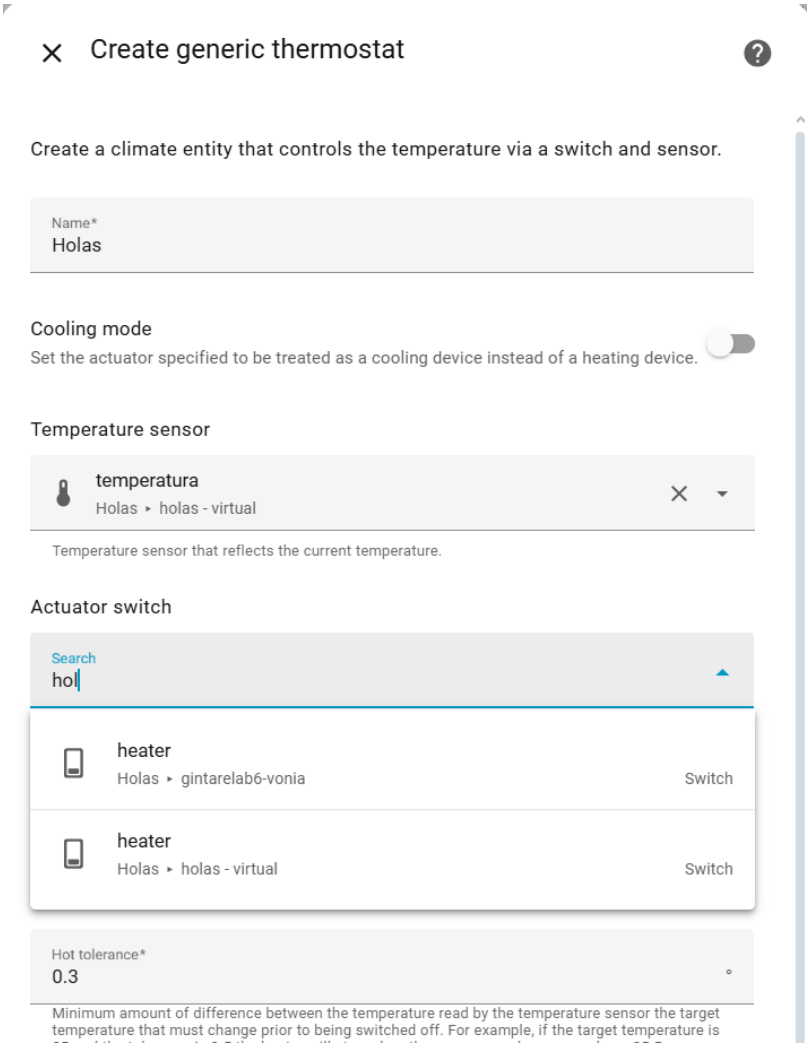
Visus virtualius termostatus aprašiau vienodai, todėl rezultatus pateiksiu tik vieno – Holo.



Pav 20 virtualHolas.yaml

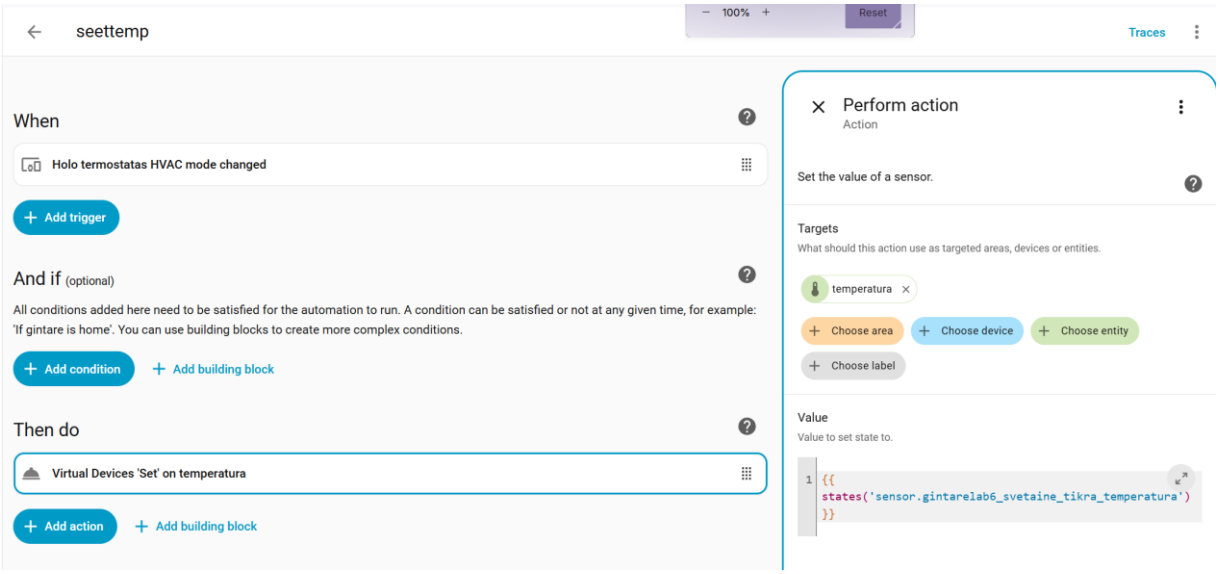
Pirmiausiai sukūriau yaml failą, kuriame aprašiau virtualų temperatūros sensorių, šildytuvą bei mygtukus jam, tam, kad jo struktūra būtų tokia kaip fizinio (iš tiesų, tų mygtukų nereikia).

Tuomet per **Settings > Devices & services > Helpers** susikūriau naują termostatą, kuris naudoja šiuos sensorius.



Pav 21 Helper virtualus termostatas

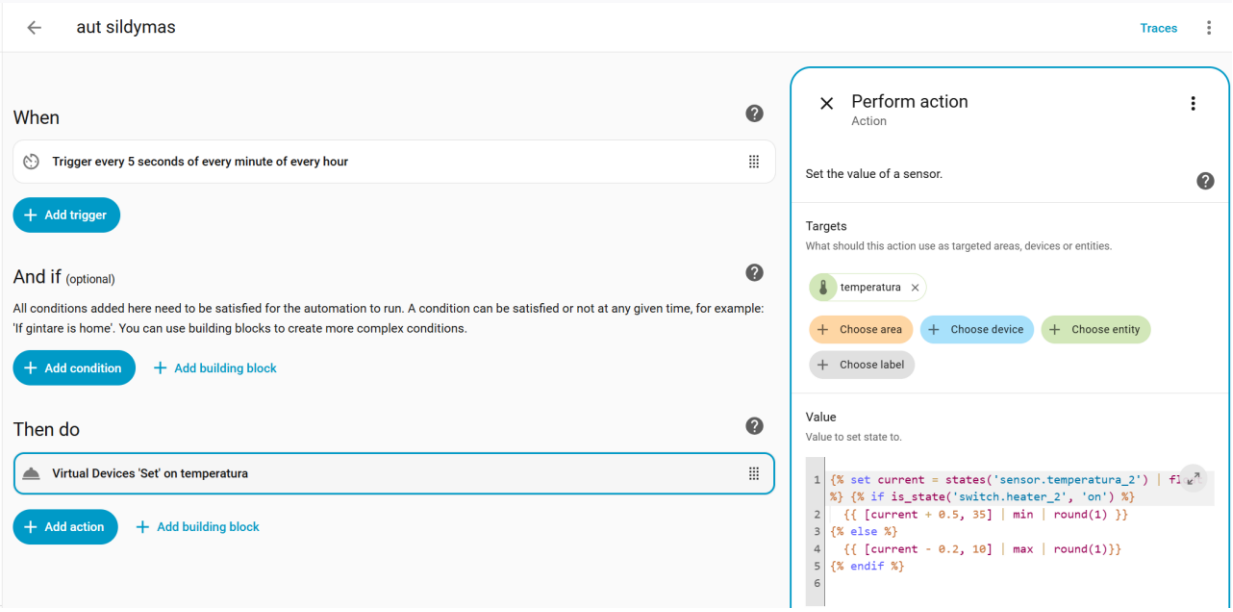
Pradinė temperatūra yra paimama iš fizinio sensoriaus, kai termostato HVAC režimas yra pakeičiamas į „heat“ iš „off“, tai reiškia kai jį įjungiame.



Pav 22 Pradinės temperatūros taisyklė

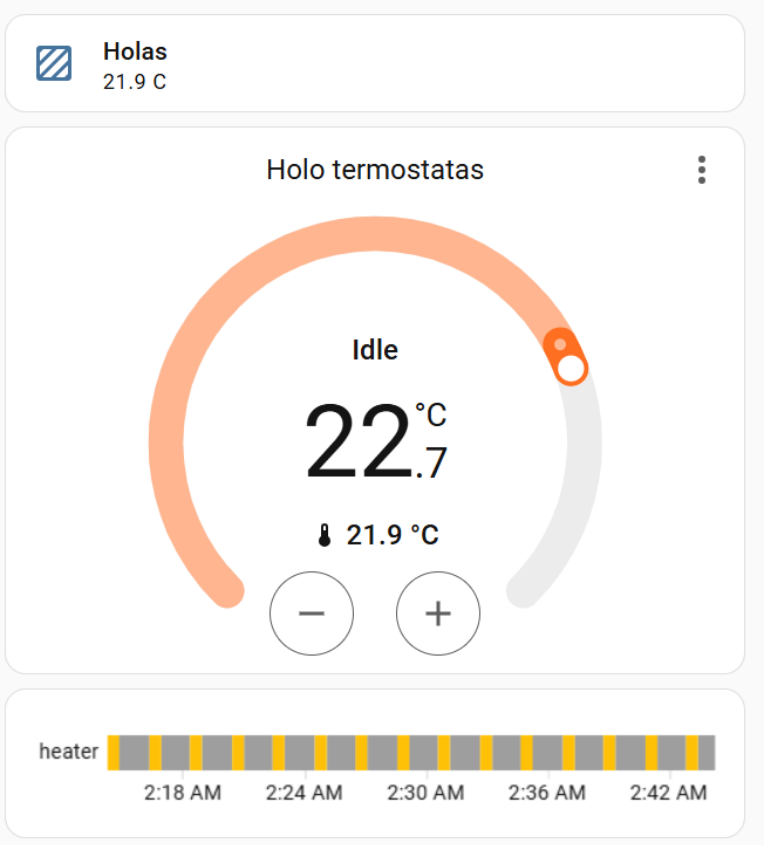
Tam, kad simuliuotume šildymą/ kambario atvėsimą buvo nustatyta 1 laipsnio Hot/Cold tolerance (reikia matyti kaip veikia testavimui todėl buvo parinktas didesnis tarpas) ir tada kodu aprašiau

temperatūros didinimą bei mažinimą: jei „heater“ yra ijungtas tai didiname pusę laipsnio, bet ne daugiau 35°C, jog apsidraustumėm nuo nesąmoningų reikšmių, o jei jis išjungtas tai mažiname pusę laipsnio, bet ne mažiau už 10°C.



Pav 23 Šildymo simuliacija

Pridėjus istorijos kortelę Overview lange, galime matyti kaip šildytuvas veikia simuliuojant kambario temperatūros kitimą:



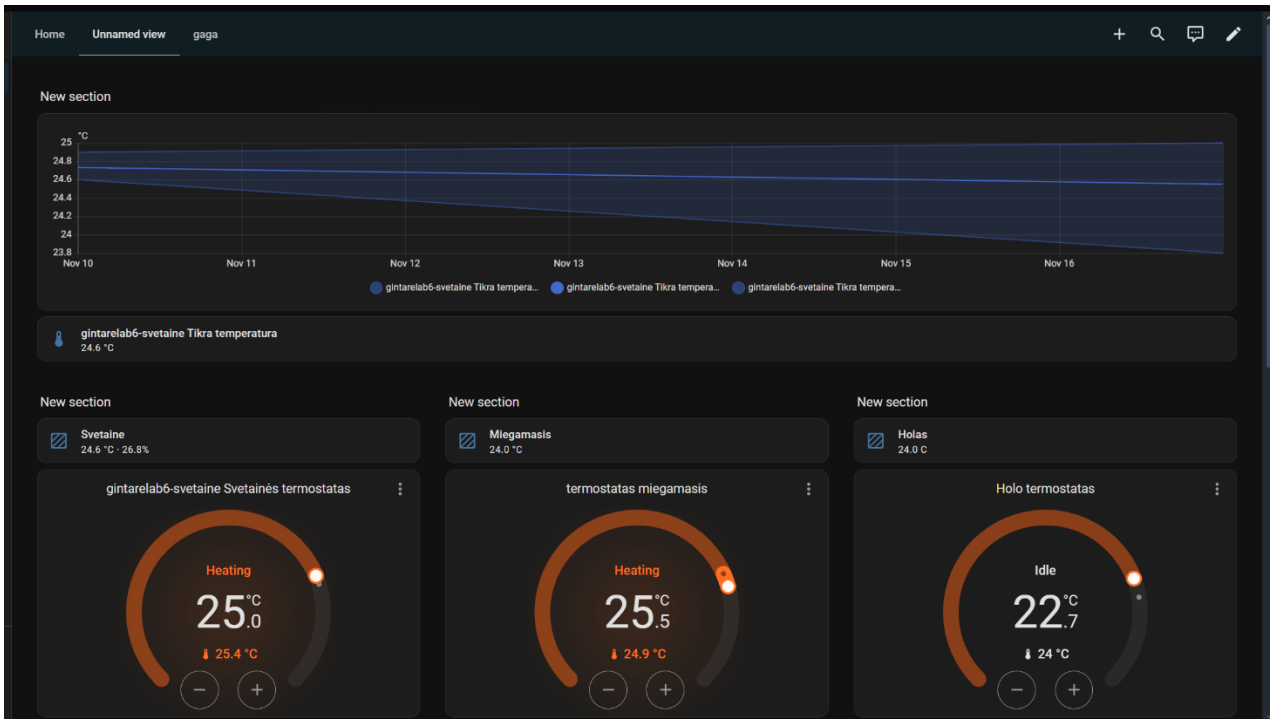
Pav 24 Holo termostatas

Svetainės šildymą taip pat reikėjo simuliuoti, todėl termostatui naudojau simuliuotą sensorių ir jo lambda aprašiau taip pat:

```
- lambda: |-
          if (!id(sim_initialized) && id(real_temp).has_state()) {
            id(sim_temp) = id(real_temp).state;
            id(sim_initialized) = true;
          }
          if (id(heater).state) {
            id(sim_temp) += 0.5;   // šildomas kambarys
          } else {
            id(sim_temp) -= 0.5;   // aušta kambarys
          }

          // ribos, kad netaptų nelogiška
          if (id(sim_temp) < 10) id(sim_temp) = 10;
          if (id(sim_temp) > 35) id(sim_temp) = 35;

          // atnaujina sensoriaus reikšmę
          id(my_temperature_sensor).publish_state(id(sim_temp));
```



Pav 25 HomeAssistant Overview



Pav 26 HomeAssistant Svetainė, miegamasis ir holas Overview



Pav 27 HomeAssistant Overview vomia ir D_kamb

2.6.3 Išvados

Ši dalis buvo kur kas sunkiau už praeitas, sudėtingiausia buvo susigaudyti tarp skirtingų įrenginių, bei teisingai simuliuoti fizinio M5Stack įrenginio termostato temperatūrą. Su Wokwi įrenginiu dirbti taip pat buvo sunku, nes jis yra labai nestabilus ir dažnai išsijungia, tada būna labai sunku jį iš naujo įjungti, nes jis neprisijungia prie interneto.

Tačiau galiausiai užduotį įgyvendinti pavyko ir viskas buvo atlikta pagal reikalavimus.

3. Bendros išvados

Atlikus visus laboratorinius darbus, pavyko sukurti veikiančią namo šildymo valdymo sistemą su įvairiais įrenginiais ir jų integracijomis. Be to susipažinau su įrenginio projektavimu tiek per EasyEDA, tiek per Wokwi simuliatoriu.

EasyEDA buvo visai paprasta, schema nubrėžti lengva, bet PCB išdėstyme reikėjo pagalvoti kaip viską sutalpinti.

Docker su HomeAssistant ir ESPHome pasirodė patogus būdas viską valdyti vienoje vietoje. Paprastas taisykles su HomeAssistant šablonais daryt buvo lengva. Bet kai reikėjo Jinja išraiškų temperatūrai keisti, kilo šiek tiek sunkumų, teko paskaityti dokumentaciją. ESPHome lambda su C++ kodu taip pat ne iš karto gavosi. HACS Virtual Components buvo daug patogesnis būdas nei per ESPHome kodą – yra daugiau šablonų ir greičiau viską nustatyt.

Laboratorijose skirto laiko užduotims pritrūko todėl teko padirbėti ir namuose, tačiau simuliavus ir pagrindinį įrenginį viską pasidaryti galiausiai pavyko ir užduotys buvo atliktos pagal reikalavimus.